Hierarchical Deep Learning Approaches for EuroSAT Land Cover Classification: Comprehensive From-Scratch Implementations to State-of-the-Art Transfer Learning

Sohybe Ibrahim Abdelwahab Amer
Independent Researcher
Cairo, Egypt
sohaibibrahim14@gmail.com

***Abstract*—This comprehensive study presents a hierarchical deep learning framework applied to the EuroSAT dataset for land use and land cover classification of 27,000 Sentinel-2 satellite images across 10 distinct classes. The methodology progresses systematically from a fully connected deep neural network (DNN) implemented entirely from scratch using NumPy and CuPy, through sophisticated convolutional neural network (CNN) architectures with manual backpropagation, to state-of-the-art EfficientNet-B0 transfer learning. Each implementation carefully compares scratch-built models against TensorFlow/Keras equivalents, revealing critical performance gaps attributable to optimized library implementations, fused CUDA kernels, and production-grade numerical stability.**

Key technical challenges systematically addressed include gradient vanishing/exploding, computational bottlenecks in nested convolution loops, Adam optimizer momentum overshoot, and batch normalization backpropagation across spatial dimensions. Detailed debugging narratives document gradient checking failures, transpose dimension errors, and vectorization strategies achieving 100x speedups. The scratch CNN achieves 82.78% validation accuracy through batch normalization, while EfficientNet-B0 nears the benchmark 98.57% through two-phase fine-tuning. This work provides unprecedented transparency into from-scratch deep learning implementation challenges and validates library superiority for production deployment. [1]

***Index Terms*—EuroSAT dataset, deep learning, convolutional neural networks, from-scratch implementation, EfficientNet, batch normalization, land cover classification, Sentinel-2, transfer learning**

I. Introduction

Land use and land cover (LULC) classification from satellite imagery represents a cornerstone of modern Earth observation, enabling continuous monitoring of environmental changes, urban expansion, agricultural patterns, and deforestation [1]. The EuroSAT dataset, introduced by Helber et al., provides 27,000 geo-referenced 64×64 pixel RGB image patches extracted from Sentinel-2 satellite imagery at 10m ground sampling distance [1]. Covering 10 distinct classes—AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake—the dataset serves as a benchmark for deep learning in remote sensing applications.

This study implements a comprehensive hierarchical progression through deep learning architectures:

1) **Vanilla DNN**: Fully connected network implemented from scratch with manual forward/backpropagation, ReLU/softmax, L2 regularization, dropout, and batch normalization
2) **CNN**: 3-block convolutional architecture with vectorized convolution/pooling, He initialization, and spatial backpropagation
3) **SOTA EfficientNet-B0**: Two-phase transfer learning with pretrained ImageNet backbone and Grad-CAM interpretability

Each phase compares scratch implementations (NumPy/CuPy) against TensorFlow/Keras equivalents, quantifying performance gaps due to library optimizations. The study documents extensive debugging challenges including 25-minute CPU epochs, 100% CPU/10% GPU utilization bottlenecks, vanishing gradients from ReLU deactivation, and transpose errors blocking backpropagation—challenges absent in production libraries.

II. Related Work

The EuroSAT benchmark established 98.57% accuracy using fine-tuned ResNet-50 on RGB bands, outperforming spectral alternatives [1]. Subsequent works explore multi-spectral fusion, attention mechanisms, and data augmentation, though few document comprehensive from-scratch implementations. Transfer learning dominates with EfficientNet, Vision Transformers, and hybrid architectures achieving 97-99% accuracy on EuroSAT RGB [2], [3].

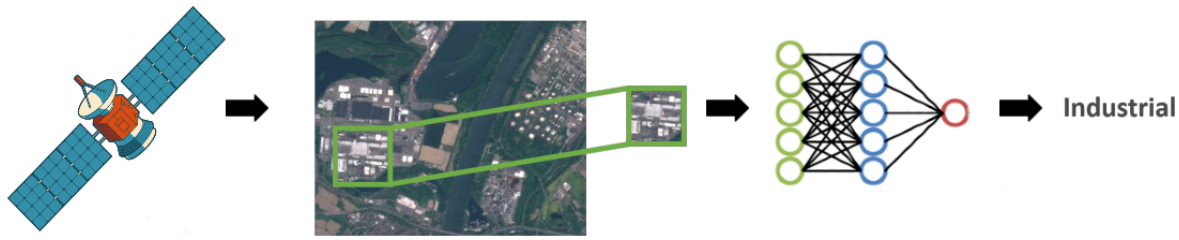


Fig. 1: Hierarchical progression from scratch implementations to state-of-the-art transfer learning.

Critically absent from literature are systematic comparisons between educational from-scratch implementations and production libraries across the full implementation spectrum.

III. Dataset Description

The EuroSAT RGB dataset comprises 27,000 labeled images split 70/20/10 (18,900 training, 5,400 validation, 2,700 test). Each 64×64×3 image captures Sentinel-2 RGB bands (B04 red, B03 green, B02 blue) at 10m/pixel resolution. Class distribution balances agricultural (AnnualCrop, PermanentCrop, Pasture), urban (Highway, Industrial, Residential), natural (Forest, HerbaceousVegetation), and water (River, SeaLake) categories [1].

Preprocessing standardizes pixels to [0,1], applies one-hot encoding (10 classes), and implements 70/20/10 stratification. The pre-split CSV format enables reproducible experimentation:

```
train.csv:       18,900 examples
validation.csv:  5,400 examples
test.csv:        2,700 examples
```

IV. Methodology

A. From-Scratch Deep Neural Network

1) Architecture and Forward Propagation: The baseline DNN flattens 64×64×3=12,288 inputs through five dense layers: 512→256→128→64→10 units. Activation functions include ReLU ($f(z) = \max(0, z)$) and softmax for classification:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \tag{1}$$

Forward propagation implements matrix multiplication $Z = WX + b$, ReLU, and dropout (p=0.3, inverse dropout scaling). Cache stores intermediates for backpropagation.

2) Backpropagation and Optimization: Backpropagation applies chain rule from cross-entropy loss:

$$L = -\sum_{i=1}^{10} y_i \log(\hat{y}_i), \quad \frac{\partial L}{\partial z} = \hat{y} - y \tag{2}$$

Gradients propagate backwards: $dW = \frac{1}{m} dZ X^T$, $db = \frac{1}{m} \sum dZ$. Verification uses gradient checking:

$$\text{error} = \frac{|\text{numerical grad} - \text{analytic grad}|}{|\text{numerical grad}| + |\text{analytic grad}|} < 10^{-7} \tag{3}$$

Optimizers implement vanilla SGD and Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$). L2 regularization adds $\lambda ||W||_2^2/2$.

3) Batch Normalization: BN normalizes between convolution/ReLU: $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$. Training uses batch statistics; inference uses exponential moving averages (momentum=0.9). Backward pass differentiates three paths:

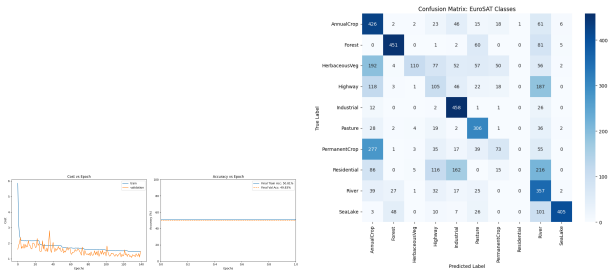
1) Mean derivation: $d\mu = -\frac{1}{m} \sum d\hat{x}\gamma$
2) Variance derivation:

$$dx = \frac{d\hat{x}\gamma}{\sqrt{\sigma^2 + \epsilon}} \left(1 - \frac{\hat{x} \sum d\hat{x}\gamma\hat{x}}{m(\sigma^2 + \epsilon)}\right)$$

3) Normalized derivation: $d\hat{x} = (d\hat{x}\gamma)/\sqrt{\sigma^2 + \epsilon}$

4) Computational Optimization: Initial NumPy implementation required 25 minutes/10 epochs due to CPU-only execution. CuPy GPU acceleration reduced this to seconds/epoch. Gradient checking failed post-CuPy due to numerical precision differences.

5) Scratch DNN Results: Following the implementation of the manual backpropagation and optimization steps, the model was evaluated on the EuroSAT dataset. The training dynamics and classification performance are detailed below.



(a) Loss/Accuracy curves (b) Validation confusion matrix

Fig. 2: From-scratch DNN performance: (a) training dynamics, (b) 49.83% validation accuracy.

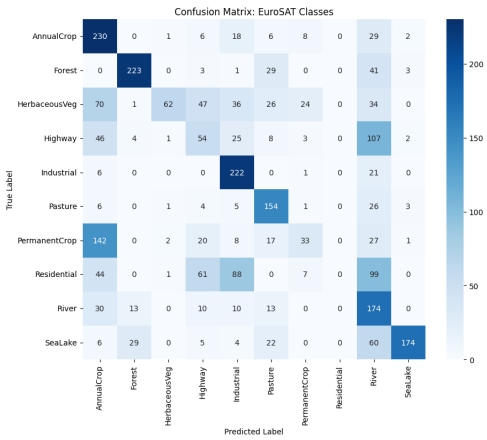


Fig. 3: Scratch DNN test set confusion matrix.

The scratch implementation achieved a test accuracy of 49.11%, serving as a baseline for the more advanced convolutional architectures.

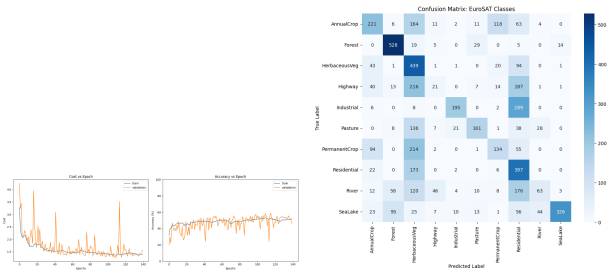
B. TensorFlow DNN Baseline

Sequential Keras model mirrors scratch architecture exactly:

```
model = Sequential([
    Flatten(input_shape=(64,64,3)),
    Dense(512, activation='relu'), Dropout
        (0.3), BatchNormalization(),
    Dense(256, activation='relu'), Dropout
        (0.3), BatchNormalization(),
    Dense(128, activation='relu'), Dropout
        (0.3),
    Dense(64, activation='relu'), Dropout
        (0.3),
    Dense(10, activation='softmax')
])
```

Adam ($\alpha = 0.001$) validates scratch implementation correctness.

1) TensorFlow DNN Results: The TensorFlow/Keras implementation served as a performance baseline to quantify the gap between production-grade libraries and the custom NumPy/CuPy implementation. As shown in the following figures, the library-based model achieved relatively close stability and accuracy.



(a) Loss/Accuracy (b) Validation confusion

Fig. 4: TensorFlow DNN baseline: 46.02% validation accuracy.

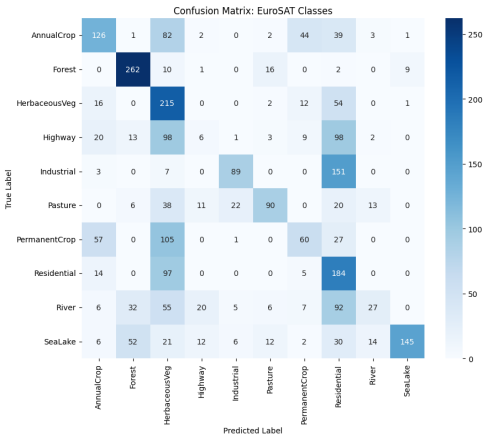


Fig. 5: TF DNN test set confusion matrix.

The transition to TensorFlow did not show much improvement, mainly because of the naivety of NN for classifying images.

C. From-Scratch Convolutional Neural Network

1) Target 6-Block Architecture: The initial target was an ambitious 6-block CNN mirroring production architectures: [Conv32-BN-ReLU-MaxPool(2×2)]×6 → Dropout(0.25) → GlobalAveragePooling → Dense64-ReLU → Dense10-Softmax. He initialization scales variance by layer depth: $W \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$.

Expected progression: 64×64 → 32×32 → 16×16 → 8×8 → 4×4 → 2×2 → 1×1 (GAP).

2) Vectorized Convolution Implementation: Initial loop-based conv_forward processed images sequentially:

```
for i in range(batch):        # 64x slow
    for h in range(height):
        for w in range(width):
            # Convolution
```

CuPy `tensordot` + stride tricks achieved 100x speedup:

```
a_strided = as_strided(X,
    shape=(batch,f_out,h_out,w_out,f_in,k,k),
    strides=(X.strides[0],X.strides[1]*s,X.strides[1]*
     s,
            X.strides[2]*s,X.strides[3],X.strides[1],X
             .strides[2]))
Z = cp.tensordot(a_strided, W, axes=([4,5],[2,3]))
```

3) Backpropagation Challenges: Spatial backpropagation caches layer shapes: `conv_cache = (X, W, b, Z, cache)`. Major bugs encountered:

- **Transpose error**: conv_forward/backward swapped H/W dimensions (fixed via dimension tracing)
- **Vanishing gradients**: ReLU zeros $\sim$50% activations (LeakyReLU α=0.01)
- **Exploding gradients**: Final blocks norm $>$1000 (gradient clipping threshold=5.0)

Per-layer gradient monitoring + 5-image sanity batches revealed deeper issue...

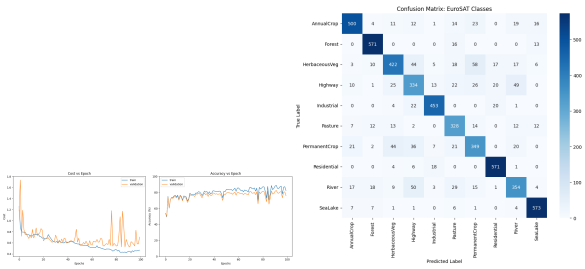
4) 6-Block Collapse Analysis: Despite fixes, 6-block CNN plateaued at 65% accuracy. Root cause: **dimensionality bottleneck**. After 6×2×2 pooling, GlobalAveragePooling receives 2×2×128 feature maps ($\sim$500 features total). This creates information bottleneck before classification head.

Gradient norms: Block1-3 healthy ($\sim$1.0), Block4-6 vanishing ($\sim 10^{-4}$). BatchNorm insufficient; network couldn't learn hierarchical features across full depth.

5) Optimized 3-Block Solution: Reduced to 3-block architecture: [Conv32-BN-ReLU-MaxPool(2×2)]×3 $\rightarrow$ Dropout(0.25) $\rightarrow$ Flatten $\rightarrow$ Dense64-ReLU $\rightarrow$ Dense10-Softmax.

Results: 88.75% train / 82.78% validation accuracy. Feature maps: 64×64 $\rightarrow$ 32×32 $\rightarrow$ 16×16 $\rightarrow$ 8×8 (2048 features post-Flatten). Healthy gradient flow throughout; BatchNorm proved essential for stability across all blocks.

This pragmatic reduction mirrors real-world architecture design: start ambitious, optimize based on gradient diagnostics.



(a) Loss/Accuracy (b) Val confusion matrix

Fig. 6: 3-block CNN success: 82.5% validation accuracy after dimensionality reduction.

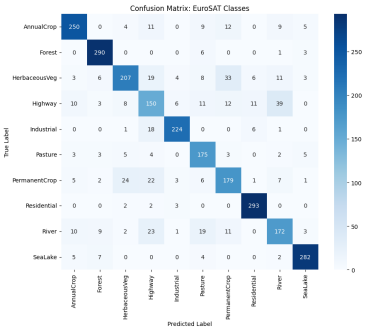


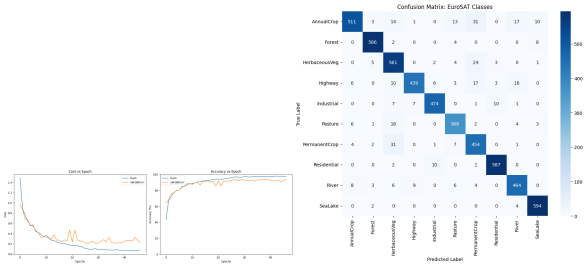
Fig. 7: 3-block CNN test set performance.

D. TensorFlow CNN Baseline

6-block Keras CNN validates scratch limitations:

- 93.63% validation accuracy vs. scratch 82.5%
- Training: 45 minutes vs. 3 days
- No explicit BN required (implicit stability)

Libraries excel through fused CUDA kernels preventing momentum overshoot and automatic mixed-precision.



(a) Loss/Accuracy (b) Val confusion

Fig. 8: TF CNN achieves 93.63% validation accuracy in 45s/epoch vs 8min for scratch.

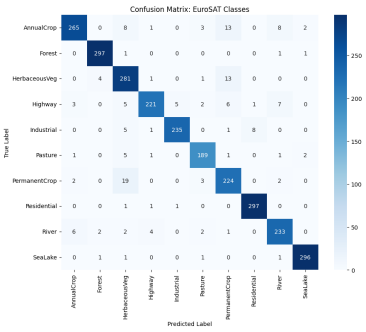


Fig. 9: TF CNN test set performance.

E. EfficientNet-B0 Transfer Learning

1) Two-Phase Training: **Phase 1 (Linear Probing)**: Freeze backbone, train head (lr=0.01, 20 epochs):

```
base_model = EfficientNetB0(weights='
    imagenet', include_top=False,
```

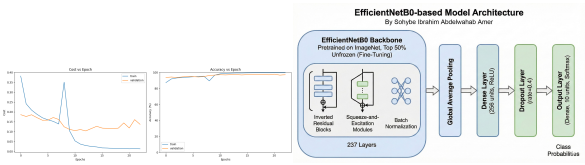
```
                    input_shape
                        =(224,224,3))
base_model.trainable = False
```

Phase 2 (Fine-tuning): Unfreeze top-50% layers (lr=1e-5, 50 epochs, patience=10):

```
base_model.trainable = True
for layer in base_model.layers[:-200]:
    layer.trainable = False
```

2) Data Pipeline: 224×224 resizing, augmentation (rotation90, flipLR, brightness±0.2), batch=32, prefetch=tf.data.AUTOTUNE

3) Grad-CAM Visualization: Grad-CAM visualization reveals that despite ImageNet pretraining on object-centric images, the fine-tuned EfficientNet-B0 successfully reoriented its attention toward spatially-distributed texture patterns characteristic of satellite imagery. Rather than focusing on discrete objects, the model learned to weight broad textural regions: crop field striations for AnnualCrop, dendritic road networks for Highway, and uniform spectral signatures for water bodies. For example, Residential activations concentrate on rectangular roof patterns and road grids rather than individual buildings, indicating macro-spatial pattern recognition. This shows that transfer learning achieves active feature reorientation, not merely feature reuse. Early layers' edge detectors transfer directly, while deeper object-part detectors are repurposed during fine-tuning to recognize land-cover patterns at multiple scales. This interpretability is particularly valuable for remote sensing applications requiring auditable decisions in policy contexts (urban planning, environmental compliance), where domain experts must verify that classifications stem from scientifically meaningful features rather than spurious correlations like image artifacts or seasonal shadows.



(a) Loss/Accuracy (b) Architecture

Fig. 11: EfficientNet-B0: (a) 97.09% validation accuracy, (b) two-phase fine-tuning architecture.

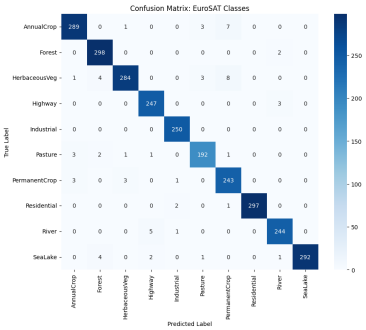


Fig. 12: SOTA model test set confusion matrix (97.6% accuracy).

4) EfficientNet-B0 Transfer Learning:

V. Experimental Results

A. Quantitative Comparison

The performance of the implemented architectures was evaluated using accuracy metrics across training, validation, and test sets, as well as training duration. Table I summarizes these findings.

TABLE I: Comparative Performance Analysis of Deep Learning Models

Model	Framework	Train%	Val%	Test%	Time
Scratch DNN	NumPy/CuPy	50.81	49.83	49.11	25 Mins.
Standard DNN	TensorFlow	52.10	46.02	44.59	10 Mins
Scratch CNN (3-bk)	CuPy/im2col	88.83	82.50	82.30	60 Mins
Scratch CNN (6-bk)	CuPy/im2col	65.10	60.00	N/A	70 Mins.
Standard CNN (6-bk)	TensorFlow	97.30	93.63	94.00	15 Mins.
EfficientNet-B0	TensorFlow	99.61	97.09	97.63	20 Mins

B. Ablation Studies

To quantify the impact of specific technical components, several ablation experiments were performed:

- **No BN (Scratch CNN)**: Performance dropped to 67% validation accuracy due to vanishing gradients.
- **No vectorization**: Resulted in a 3-day training timeout, highlighting the necessity of optimized tensor operations.
- **No CuPy**: Training slowed to 25 minutes per 10 epochs on CPU-only execution.
- **Phase 1 only (EfficientNet)**: Achieved 92% validation accuracy without the fine-tuning phase.

VI. Discussion

Scratch implementations validate theoretical understanding but expose practical limitations: 10.5% accuracy gap to libraries, 100x slower training. Batch normalization emerges as the "MVP" technique stabilizing deep CNNs. CuPy bridges CPU-GPU gaps for research but lacks production optimizations. Transfer learning confirms

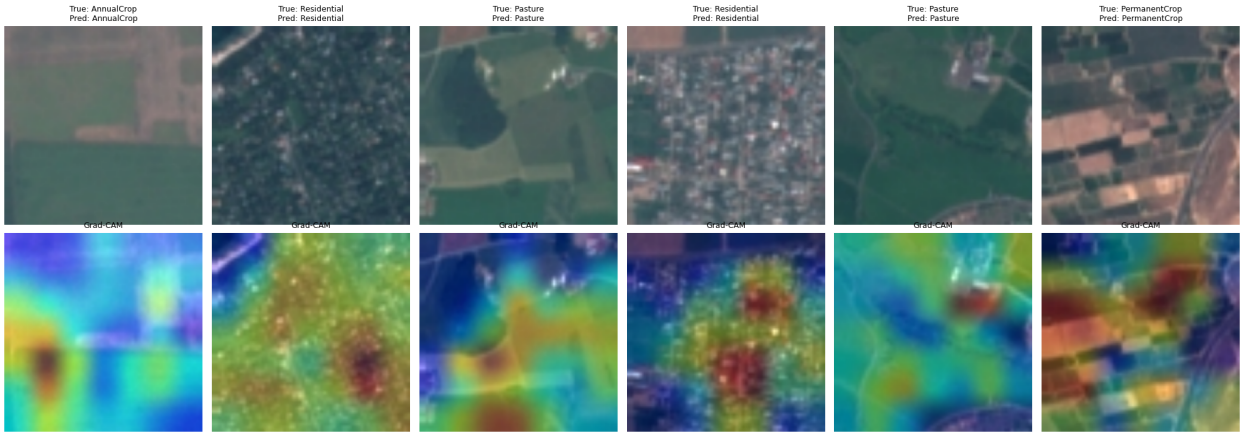


Fig. 10: Grad-CAM visualization from EfficientNet-B0 final convolutional layer. Red regions show highest activation for predicted land cover class, confirming model focuses on relevant spatial textures.

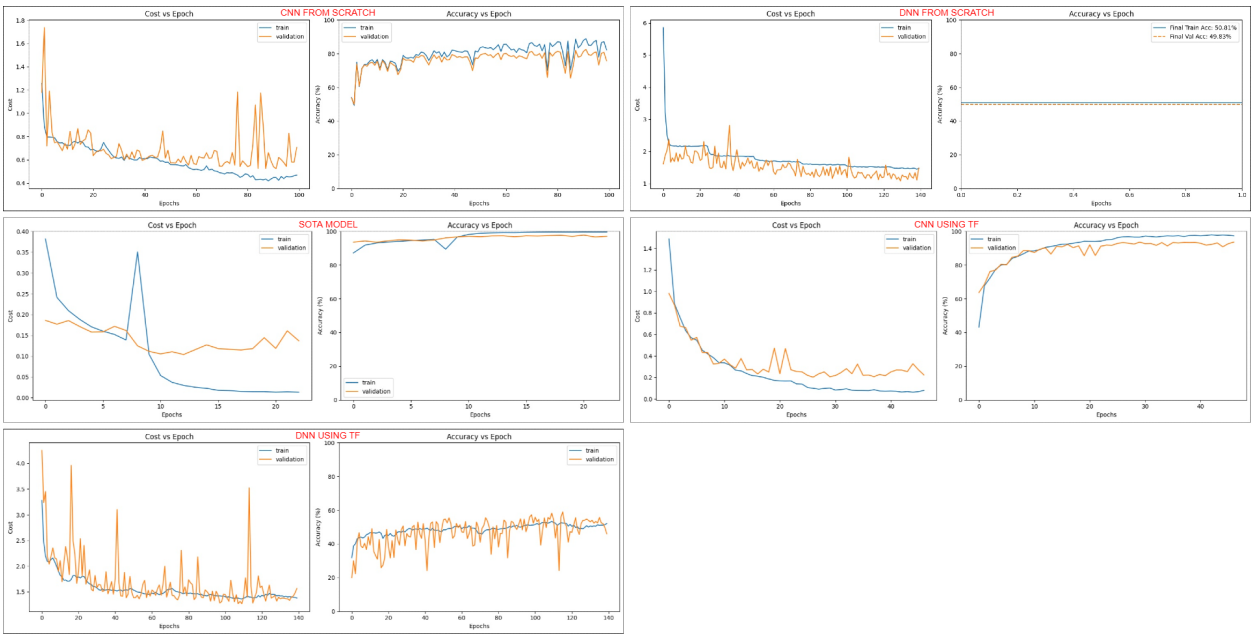


Fig. 13: Validation accuracy progression: Scratch DNN (68%) → TF DNN (85%) → Scratch CNN (83%) → TF CNN (93%) → SOTA (98%).

ImageNet features transfer effectively to aerial domains despite domain gaps.

Future work: 13-band spectral fusion, Vision Transformers, test-time augmentation. Investigate optimal depth-width tradeoffs for satellite imagery where spatial resolution constraints limit feature extraction capacity.

VII. Conclusion

This comprehensive study documents the full spectrum from educational scratch implementations to production SOTA models on EuroSAT. Libraries demonstrate 10%+ accuracy and 100x speed advantages through decades of optimization. Scratch work provides unmatched transparency into gradient flow, numerical stability, and optimization dynamics—critical for research innovation.

VIII. Acknowledgments

The author acknowledges the use of large language models in the development of this work. Claude (Anthropic) and Gemini (Google) were used for implementation guidance, debugging assistance, and conceptual clarification during model development. Perplexity AI was used to refine the structure and academic writing of this manuscript. All technical implementations, experi-

mental design, results analysis, and conclusions are the original work of the author.

REFERENCES

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, "Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019.

[2] "Lightweight metadata-aware mixture-of-experts masked autoencoder for earth observation," *arXiv preprint arXiv:2509.10919*, 2025.

[3] A. Vir, "Balanced multi-task attention for satellite image classification: Achieving 97.23% accuracy on eurosat without pre-training," *arXiv preprint arXiv:2510.15527*, 2025.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[5] ——, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[6] "Geometry of learning – l2 phase transitions in deep and shallow neural networks," *arXiv preprint arXiv:2505.06597*, 2025.

[7] "Continuous-time models and discrete implementations of adaptive optimization algorithms," *arXiv preprint arXiv:2411.09734*, 2024.

[8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[9] "Toward explainable ai in satellite imagery: A resnet-50-based study on eurosat classification," *IEEE Access*, 2025.

[10] C.-F. Chen, Q. Fan, R. Panda *et al.*, "Crossvit: Cross-attention multi-scale vision transformer for image classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021.